

LOCK 'N' BREATHE

Submitted By:

Members Involved:

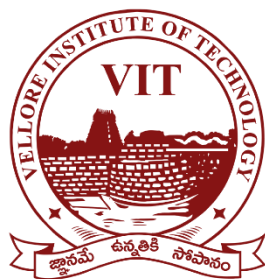
<u>NAME</u>	<u>REG. NUMBER</u>
Ananya Jaiswal	24BCA7023
Samiran Saha	24BCE7949
Yarramsetty Niharika	24BCE7184
Akshat Agrawal	24BCA7140
AV Spoorthi	24BCE7877
Saraswathi Thanusha	24BCE7732

Under the Guidance of:

Ms. Yarlagadda Siva Reshma

Assistant Professor Junior

School of Computer Science and Engineering (SCOPE)



VIT-AP
UNIVERSITY
Apply Knowledge. Improve Life!®

ABSTRACT

This project, titled Lock 'n' Breathe: Alcohol Detection System, focuses on improving road safety by preventing drunk driving through automation and embedded technology. Drunk driving has become one of the leading causes of road accidents and fatalities, especially among commercial vehicle drivers in India. Despite strict traffic regulations, incidents involving intoxicated drivers continue to rise, creating an urgent need for an effective preventive mechanism.

The proposed system detects alcohol concentration in a driver's breath before ignition and prevents the vehicle from starting if the level exceeds the permissible limit. It uses an MQ-3 alcohol sensor to sense alcohol vapors, an Arduino microcontroller to process the data, and a relay mechanism to disable the ignition circuit. Additional components such as an LCD display, buzzer, and GPS module enhance the system's functionality, while a keypad override feature allows authorized access during emergencies.

The prototype was successfully designed, developed, and tested, showing high accuracy and quick response in detecting alcohol levels. Its compact and low-cost design makes it suitable for integration into both existing and new vehicles. Overall, this project provides a practical and reliable solution to reduce drunk driving incidents and promote responsible driving behavior through the use of modern automation and embedded system technology.

INDEX

Introduction.....	1
Background.....	2
Problem Definition.....	2
Objectives of the Proposed Work.....	3
Methodology.....	3
• System Components.....	3
• Working	5
• Prototype.....	5
• Flowchart.....	6
Results and Discussion.....	7
Conclusion and Future Scope.....	7
• Conclusion	7
• Future Scope	8
References.....	8
Codes in Appendix.....	9

LIST OF TABLES:

- Table 1: Alcohol Detection System Output Based on Alcohol Concentration Levels

LIST OF FIGURES:

- Figure 1: Labelled Diagram of the Arduino Uno R4 Wi-Fi microcontroller
- Figure 2: Labelled Diagram of MQ-3 Alcohol Sensor
- Figure 3: Connection Diagram of the 16X2 LCD
- Figure 4: Labelled Diagram of NEO-6M GPS Module
- Figure 5: Labelled Diagram of the SD card Module
- Figure 6: Pin Diagram of the Relay Module
- Figure 7: Prototype of Lock 'n' Breathe Alcohol Detection Engine locking system
- Figure 8: Flowchart of the working of the prototype

ABBREVIATIONS:

1. LCD – Liquid Crystal Display
2. I²C – Inter-Integrated Circuit
3. MQ-3 – Metal Oxide Semiconductor Alcohol Sensor
4. SD – Secure Digital (Memory Card)
5. GPS – Global Positioning System
6. ppm – Parts Per Million

INTRODUCTION

Road safety has become a major global concern as the number of accidents caused by human negligence continues to rise. Among these, drunk driving is one of the leading factors responsible for road fatalities and severe injuries. In India, 4,201 people died in drunk driving-related accidents in 2022 and around 273,000 people die per year due to the alcohol consumption. Alcohol consumption impairs a driver's judgment, reaction time, and coordination, making it extremely dangerous to operate a vehicle under its influence. To ensure safety on roads, modern technology has become a key tool for developing smarter and more efficient preventive systems that minimize the risks caused by human error.

The relevance of vehicle safety and automation in today's world is higher than ever. As transportation systems expand and the number of vehicles increases, the need for intelligent and automated safety mechanisms grows. Traditional enforcement methods such as manual alcohol testing and traffic monitoring are limited in scope and cannot provide continuous real-time safety assurance. Therefore, integrating technology-based solutions within vehicles has become an essential step toward achieving safer roads and reducing accident rates.

This project introduces an Automatic Alcohol Detection System that aims to prevent drunk driving by detecting alcohol levels in a driver's breath before the vehicle starts. The system is designed to automatically sense alcohol concentration using a gas sensor and disable the ignition when the detected level exceeds the permissible limit. In addition, it provides alert features to notify the driver and nearby individuals, ensuring preventive action is taken before any risk occurs. The system functions independently without human supervision, making it reliable, efficient, and suitable for integration into modern vehicles.

The main vision of this project is to contribute to safer and smarter transportation by leveraging embedded systems and automation technology. It offers a practical and low-cost solution to one of the most common causes of road accidents — driving under the influence of alcohol. By preventing the vehicle from starting when the driver is intoxicated, this project helps protect lives, reduce accidents, and promote responsible driving behavior. Ultimately, it demonstrates how engineering innovation can be used to solve real-world social challenges and support national road safety initiatives.

BACKGROUND

Road safety has become a critical issue worldwide, with drunk driving being one of the leading causes of road accidents and fatalities. Alcohol consumption affects the driver's mental alertness, coordination, and decision-making ability, making vehicle operation highly unsafe. In India, thousands of lives are lost every year due to alcohol-influenced driving despite strict laws and awareness programs. Such incidents also lead to severe economic losses and emotional suffering for affected families.

Current preventive measures like manual breath analyzers and traffic checkpoints are limited in reach, time-consuming, and dependent on human involvement. These methods are reactive, identifying offenders only after they have started driving. The lack of automated systems within vehicles for early detection of intoxicated drivers continues to be a major safety gap. With the growth of technology, there is an urgent need for a reliable, affordable, and automatic system that can prevent vehicles from starting when the driver is under the influence of alcohol. To address this issue, the Lock 'n' Breathe project introduces an intelligent alcohol detection system that uses sensors and automation to detect alcohol in the driver's breath and prevent ignition, ensuring safer and more responsible driving.

PROBLEM DEFINITION

The proposed project addresses a major gap in vehicle safety — the lack of an automated mechanism to prevent an intoxicated person from starting a vehicle. While existing laws and manual enforcement methods attempt to control drunk driving, they remain reactive and heavily dependent on human intervention. Most vehicles still lack built-in systems capable of detecting alcohol consumption before ignition, resulting in preventable accidents.

This project aims to design and develop an automatic alcohol detection system integrated with a vehicle's ignition control to ensure proactive prevention. The system employs an MQ-3 gas sensor to detect alcohol concentration in the driver's breath and an Arduino microcontroller to process the data. When the detected level exceeds the permissible limit, a relay-based mechanism disables the ignition while a buzzer and LCD display provide immediate alerts.

Additional features such as a GPS module for emergency location sharing and a password override option for authorized access enhance the system's reliability and usability. The scope of this project is limited to pre-ignition detection and engine control, excluding post-ignition monitoring. Within these defined boundaries, the Lock 'n' Breathe system offers a low-cost, reliable, and efficient solution to prevent drunk driving and improve road safety.

OBJECTIVE OF THE PROPOSED WORK

The main objective of the proposed work is to develop a smart alcohol detection and vehicle control system that enhances road safety by preventing drunk driving. The system ensures that the vehicle ignition only activates if the driver's breath alcohol concentration is within safe limits

METHODOLOGY

- **SYSTEM COMPONENTS:**

The hardware components used in the Lock 'n' Breathe system are as follows:

- Arduino Uno R4 Wi-Fi: This acts as the main controller of the system. It reads data from sensors, processes inputs, and controls outputs like the relay, motor, and buzzer.

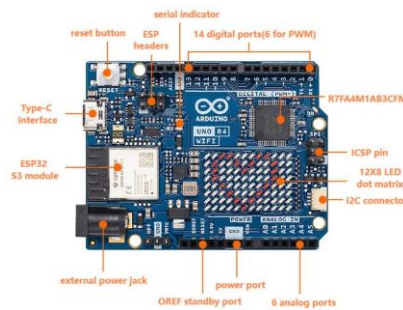


Figure 1

- MQ-3 Alcohol Sensor: It detects alcohol concentration from the driver's breathe and sends an analog signal to the Arduino for processing.

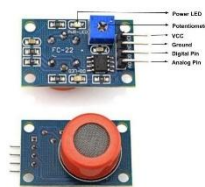


Figure 2

- 16x2 LCD (I²C): Displays system messages such as alcohol level, engine status, and alerts to the driver.

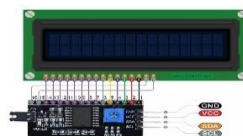


Figure 3

- iv. GPS Module (NEO-6M): Provides the location of the vehicle, which is recorded during detection events.

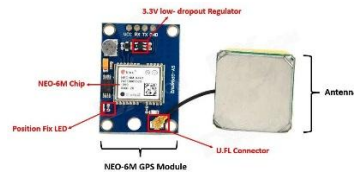


Figure 4

- v. SD Card Module: It stores data such as alcohol readings, GPS location, and event logs.



Figure 5

- vi. Relay Module: It acts as a switch to control the motor or engine ignition based on alcohol level.



Figure 6

- vii. Buzzer: Provides an audible warning when unsafe alcohol levels are detected.
- viii. 4×4 Keypad: Used for password input when the system detects a moderate alcohol level.
- ix. DC Motor: Represents the vehicle engine in the prototype. It runs only when the alcohol level is safe.
- x. Li-ion Battery: A 3.7 V rechargeable battery powers the entire system.

Note: - The software for this project was developed using the Arduino IDE, which programs the Arduino Uno to control sensors, display units, and communication modules. Various libraries were utilized to enable smooth operation. These libraries collectively ensure efficient sensor data

processing, real-time monitoring, and reliable communication with the ThingSpeak IoT platform for remote tracking and analysis.

- **WORKING:**

- i. When the system is powered on, the Arduino Uno initializes all connected components, including the MQ-3 alcohol sensor, LCD display, GPS module, Wi-Fi, and SD card. The LCD displays a welcome message indicating that the system is ready. The MQ-3 sensor undergoes a short warm-up and calibration process to ensure accurate readings.
- ii. The driver is then prompted to blow onto the sensor, which detects the alcohol concentration in parts per million (ppm) from the breath sample. The Arduino processes the sensor data and compares it with predefined threshold values to determine the driver's condition. If the reading is 1- 100 ppm, the LCD displays "SAFE: X ppm" and the engine is unlocked. For readings between 100 ppm and 299 ppm, the system displays a low-level warning but still allows the engine to start. If the reading falls between 300 ppm and 499 ppm, the system activates a 30-second countdown timer for password bypass—allowing the engine to start only if the correct password is entered; otherwise, the engine remains locked and a buzzer is activated. For readings of 500 ppm or above, the bypass is denied, the buzzer sounds, and the engine stays locked. If the driver does not blow onto the sensor, then sensor will show 0 ppm. In another case, if the driver does not press 1 during bypass, then the engine will automatically lock and again sensor restart again.
- iii. In each case, the system logs the alcohol reading, alcohol level (e.g.: safe, low, medium, high) to the SD card and GPS module fetches the latitude and longitude with date and time only in the case of medium level (both the cases if password is correct or wrong) and high level and uploads the latitude, longitude and alcohol level to the ThingSpeak IoT platform for remote monitoring. Throughout the process, the LCD provides real-time messages guiding the user and ensuring that only sober drivers can start the vehicle.

- **PROTOTYPE:**

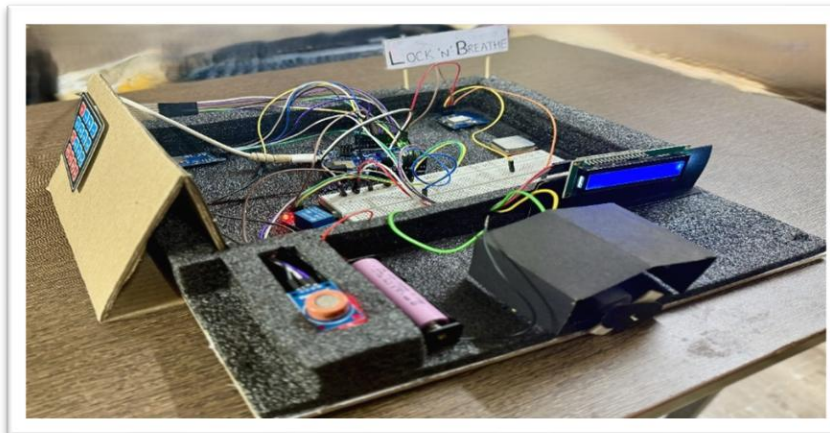


Figure 7

- **FLOWCHART:**

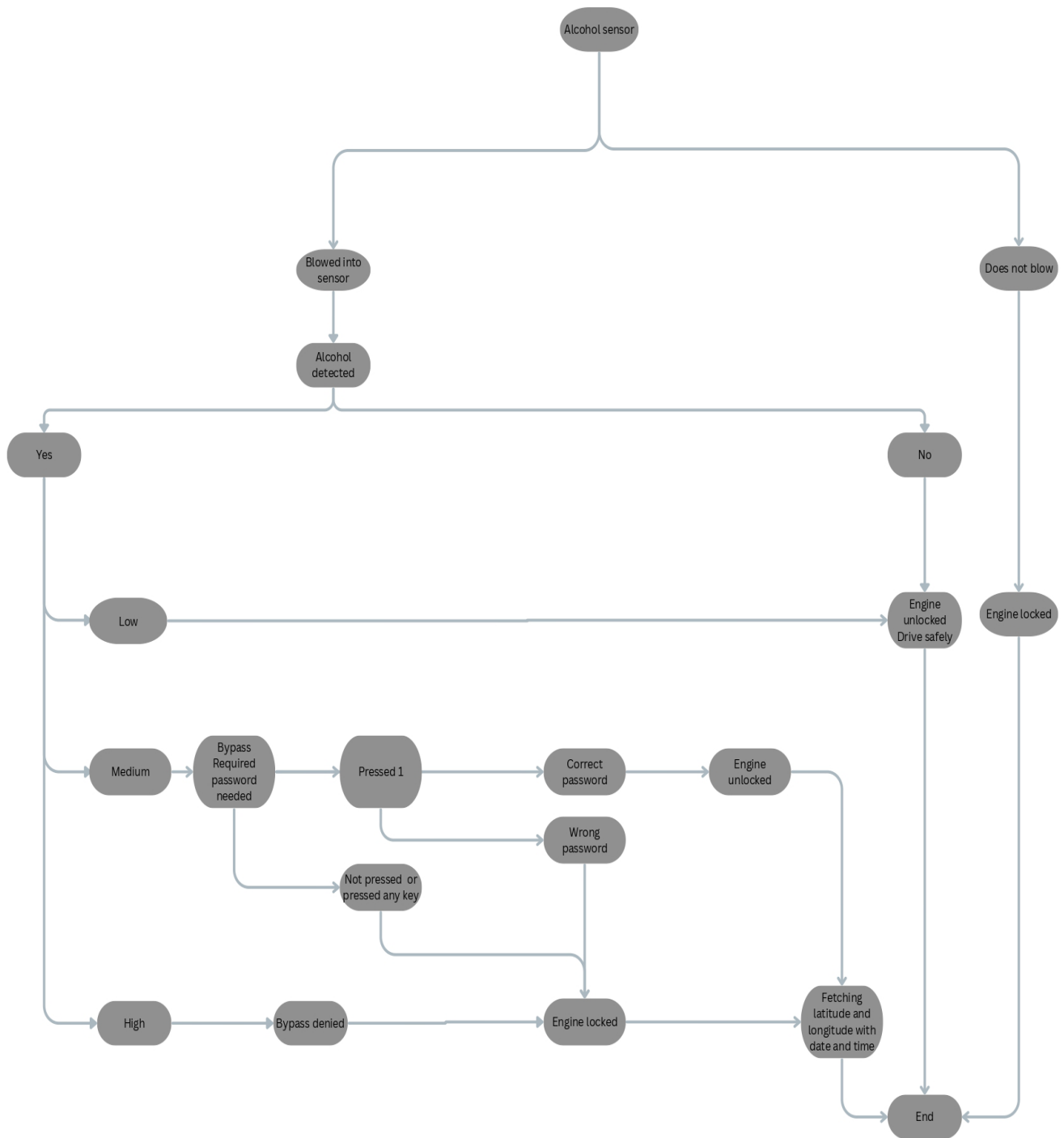


Figure 8

RESULT AND DISCUSSION

During experimentation, the prototype exhibited distinct functional outcomes for different alcohol concentration levels, confirming its ability to classify and respond accurately to varying input conditions.

Condition	PPM Range	LCD Display Message	System Action	Additional Response	Latitude/Longitude Displayed	Data Logging Label
No Blow	0 ppm	Not blow into sensor	Engine locked	normal	Nil	Nil
SAFE	1-99 ppm	“SAFE: X ppm” “No alcohol detected” “Engine unlocked, Drive safe”	Engine unlocked; vehicle can start	No buzzer; normal operation	Not displayed	SAFE
LOW	100 – 299 ppm	“LOW: X ppm” “Range: 100–299 ppm” “Engine unlocked, Drive safe”	Engine unlocked; vehicle can start	No buzzer; normal operation	Not displayed	LOW
MEDIUM	300 – 499 ppm	“MEDIUM: X ppm” Countdown timer (30s) Prompt: “Press 1 for bypass” if not pressed then engine automatically locked, again it will ask to blow	If password correct → engine unlocks If incorrect or no password → engine locked	Buzzer activates; LCD shows date, time, latitude & longitude	Displayed	MEDIUM (Bypassed) / MEDIUM (Bypass Failed)
HIGH	≥ 500 ppm	“HIGH: X ppm” “Range: above 500 ppm” “Bypass denied” “Engine locked”	Engine locked; ignition disabled	Continuous buzzer; LCD shows date, time, latitude & longitude	Displayed	HIGH

Table 1

CONCLUSION & FUTURE SCOPE

CONCLUSION:

The Lock 'n' Breathe system accurately detects alcohol levels in a driver's breath and controls the vehicle's ignition to ensure safe operation. By integrating the MQ-3 sensor, NEO-6M GPS module, relay mechanism, LCD display, SD card logging, and ThingSpeak cloud, it enables real-time monitoring and location tracking. The system records and transmits data effectively, providing both local and remote visibility. Overall, the prototype reliably prevents drunk driving and offers a practical, low-cost solution to enhance road safety.

FUTURE SCOPE:

- a) Using more advanced alcohol sensors for higher accuracy and faster response.
- b) Adding mobile app integration for real-time notifications and alerts.
- c) Vehicle speed monitor with upper speed limit under emergency bypass.
- d) Unlocking with face recognition feature with special permission add-on.

REFERENCES

- a. MQ-3 Alcohol Sensor Datasheet: <https://cdn.sparkfun.com/assets/6/a/1/7/b/MQ-3.pdf>
- b. Arduino Official Documentation: <https://docs.arduino.cc/>
- c. 4X4 Matrix Keypad: <https://www.electronicwings.com/sensors-modules/4x4-keypad-module>
- d. ThingSpeak Sign-In Page: <https://thingspeak.com/login>
- e. Micro SD Card Module for Arduino: <https://components101.com/modules/micro-sd-card-module-pinout-features-datasheet-alternatives>
- f. Neo 6M GPS Module: <https://randomnerdtutorials.com/guide-to-neo-6m-gps-module-with-arduino/>
- g. Fiore, J. (2019). Embedded Controllers using C and Arduino + Lab Manual. Mohawk Valley: Open Textbooks
- h. Arduino in Science: Collecting, Displaying, and Manipulating Sensor Data (Richard J. Smythe, 2021).

APPENDIX: SOURCE CODE

```

1 #include <Wire.h>
2 #include <hd44780.h>
3 #include <hd44780_I2Cexp.h>
4 #include <WiFi.h>
5 #include <ArduinoJson.h>
6 #include <RTC.h>
7 #include <SD.h>
8 const int SD_CS_PIN = 0;
9 int logCounter = 1;
10 hd44780_I2Cexp lcd;
11 char ssid[] = ""; //give your ssid name (hotspot name)
12 char pass[] = ""; //give your hotspot password
13 WiFiClient client;
14 #define MQ3_PIN A2
15 #define RELAY_PIN 8
16 #define BUZZER_PIN 10
17 bool f = true;
18 int baselineReading = 0;
19 const int SAFE_THRESHOLD = 100;
20 const int LOW_THRESHOLD = 300;
21 const int RED_THRESHOLD = 500;
22 const byte ROWS = 4;
23 const byte COLS = 4;
24 char keys[ROWS][COLS] = {
25   {'1','2','3','A'},
26   {'4','5','6','B'},
27   {'7','8','9','C'},
28   {'*','0','.','D'}
29 };
30 byte rowPin[ROWS] = {2, 3, 4, 5};
31 byte colPin[COLS] = {6, 7, A0, A1};
32 String correctPassword = "7800";
33 bool engineInLocked = false;
34 bool onTrack = false;
35 int lockedAlcoholValue = -1;
36 const char* THINGSPEAK_SERVER = "api.thingspeak.com";
37 const char* THINGSPEAK_WRITE_API_KEY = "HD68QVNZ2M0D14PK";
38 char getKeyPressed_manual() {
39   for (int r = 0; r < ROWS; r++) {
40     digitalWrite(rowPin[r], LOW);
41     delay(5);
42     for (int c = 0; c < COLS; c++) {
43       if (digitalRead(colPin[c]) == LOW) {
44         delay(20);
45         if (digitalRead(colPin[c]) == LOW) {
46           char pressedKey = keys[r][c];
47           while (digitalRead(colPin[c]) == LOW);
48           digitalWrite(rowPin[r], HIGH);
49           return pressedKey;
50         }
51       }
52     }
53     digitalWrite(rowPin[r], HIGH);
54     delay(5);
55   }
56   return 0;
57 }
58 void initializeSystem() {
59   lcd.clear();
60   lcd.print("Welcome to ");
61   lcd.setCursor(0, 1);
62   lcd.print("LOCK 'm BREATHE'");
63   delay(4000);
64   lcd.clear();
65   lcd.print("System is ");
66   lcd.setCursor(0, 1);
67   lcd.print("initializing...");
68   delay(3000);
69 }
70 void decide() {
71   lcd.clear();
72   lcd.print("Press A to Start");
73   lcd.setCursor(0, 1);
74   lcd.print("Press B for data");
75   while (true) {
76     char d = getKeyPressed_manual();
77     if (d == 'A') {
78       lcd.clear();
79       warmupMQ3();
80       break;
81     } else if (d == 'B') {
82       lcd.clear();
83       lcd.print("Displaying data...");
84       lcd.clear();
85       displayLogData();
86       if (f == false) {
87         lcd.clear();
88         lcd.print("Press A to Start");
89         lcd.setCursor(0, 1);
90         lcd.print("Press B for data");
91         delay(1000);
92         continue;
93       }
94     }
95   }
96 }
97 void warmupMQ3() {
98   lcd.clear();
99   lcd.print("Please wait for");
100   lcd.setCursor(0, 1);
101   lcd.print("60 seconds");
102   delay(1000);

```

```

103   for (int i = 60; i > 0; i--) {
104     lcd.clear();
105     lcd.print("warming sensor");
106     lcd.setCursor(0, 1);
107     lcd.print("wait: ");
108     lcd.print(i);
109     delay(1000);
110   }
111   lcd.clear();
112   lcd.print("Calibrating...");
113   lcd.setCursor(0, 1);
114   lcd.print("Don't blow yet!");
115   delay(3000);
116   long total = 0;
117   for (int i = 0; i < 10; i++) {
118     total += analogRead(MQ3_PIN);
119   }
120   delay(100);
121   int avgReading = total / 10;
122   baselineReading = avgReading * 0.1023 * 0.1000;
123   lcd.clear();
124   lcd.print("Sensor ready");
125   delay(3000);
126   Serial.print("Baseline: ");
127   Serial.print(baselineReading);
128   Serial.println(" ppm");
129   bool waitforTime() {
130     lcd.clear();
131     const char* message = "Please blow into the sensor: ";
132     int messageLength = strlen(message);
133     int scrollPosition = 0;
134     const int LCD_WIDTH = 16;
135     unsigned long startTime = millis();
136     int maxReading = 0;
137     while ((millis() - startTime) < 10000) {
138       lcd.setCursor(0, 0);
139       for (int i = 0; i < (LCD_WIDTH - 1); i++) {
140         int charIndex = (scrollPosition + i) % messageLength;
141         lcd.print(message[charIndex]);
142       }
143       scrollPosition++;
144       if (scrollPosition >= messageLength) {
145         scrollPosition = 0;
146       }
147       int reading = analogRead(MQ3_PIN);
148       if (reading > maxReading) {
149         maxReading = reading;
150       }
151       int remaining = 10 - ((millis() - startTime) / 1000);
152       lcd.setCursor(0, 1);
153       lcd.print("Time left: ");
154       lcd.print(remaining);
155       lcd.print(" ");
156       delay(100);
157     }
158     int rowPM = avgReading * 0.1023 * 0.1000;
159     lockedAlcoholValue = rowPM - baselineReading;
160     if (lockedAlcoholValue < 0) {
161       lockedAlcoholValue = 0;
162     }
163     lcd.clear();
164     lcd.print("Reading alcohol");
165     lcd.setCursor(0, 1);
166     lcd.print("Level...");
167     delay(2000);
168     Serial.print("Alcohol: ");
169     Serial.print(lockedAlcoholValue);
170     Serial.println(" ppm");
171     return true;
172   }
173   bool sendToThingspeak(float latitude, float longitude, int alcoholPM) {
174     WiFiClient tcpClient;
175     if (!tcpClient.connect(THINGSPEAK_SERVER, 80)) {
176       Serial.println("Thingspeak connection failed");
177       return false;
178     }
179     String postdata = String("fields=" + latitude + "&fields=" + longitude + "&fields=" + alcoholPM);
180     String httprequest = String("POST /api/v1/fields/keys/" + THINGSPEAK_WRITE_API_KEY + " HTTP/1.1\n" +
181       "Host: " + THINGSPEAK_SERVER + "\n" +
182       "Content-Type: application/x-www-form-urlencoded\n" +
183       "Content-Length: " + postdata.length() + "\n" +
184       postdata);
185     tcpClient.print(httprequest);
186     unsigned long timeout = millis();
187     while (!tcpClient.connected() && (millis() - timeout) < 5000) {
188       while (!tcpClient.available()) {
189         String line = tcpClient.readStringUntil('\n');
190         Serial.println(line);
191       }
192     }
193     tcpClient.stop();
194     Serial.println("Data sent to Thingspeak!");
195     return true;
196   }
197   void displayDateTime() {
198     lcd.clear();
199     lcd.print("Fetching data");
200     lcd.setCursor(0, 1);
201     lcd.print(" and Time: ");

```

```

202   delay(1000);
203   WiFiClient httpClient;
204   Serial.println("\nTrying HTTP time API...");
205   unsigned long epochTime = WiFi.getTime();
206   if (epochTime == 0) {
207     Serial.println("WiFi failed, retrying...");
208     delay(1000);
209     epochTime = WiFi.getTime();
210   }
211   if (epochTime == 0) {
212     Serial.println("ERROR: Could not get time");
213     lcd.clear();
214     lcd.print("Time fetch");
215     lcd.setCursor(0, 1);
216     lcd.print("failed");
217     delay(2000);
218     return;
219   }
220   Serial.print("Got epoch time: ");
221   Serial.println(epochTime);
222   String timeZoneStr = "Asia/Kolkata";
223   int offsetSeconds = 19800;
224   if (!httpClient.connect("ip-api.com", 80)) {
225     Serial.println("Connected to ip-api.com");
226     httpClient.print("GET /json?lat=12.916666&offset=19800 HTTP/1.1");
227     httpClient.print("Host: ip-api.com");
228     httpClient.print("Connection: close");
229     delay(500);
230     unsigned long timeout = millis();
231     while (!httpClient.available() && (millis() - timeout) < 10000) {
232       delay(10);
233     }
234     if (httpClient.available()) {
235       Serial.println("Got response from ip-api");
236       while (httpClient.available()) {
237         String line = httpClient.readStringUntil('\n');
238         if (line == "\n") break;
239       }
240       String response = "";
241       while (httpClient.available()) {
242         response += (char)httpClient.read();
243       }
244       httpClient.stop();
245       Serial.println("Response: " + response);
246       int tIndex = response.indexOf("timezone");
247       if (tIndex != -1) {
248         String timeZone = response.substring(tIndex + 12, tIndex + 50);
249         int endQuote = timeZone.indexOf("\"");
250         timeZoneStr = timeZone.substring(0, endQuote);
251         Serial.println("timezone: " + timeZoneStr);
252       }
253       int offsetIndex = response.indexOf("offset");
254       if (offsetIndex != -1) {
255         String offsetStr = response.substring(offsetIndex + 9, offsetIndex + 20);
256         int comma = offsetStr.indexOf(",");
257         if (comma == -1) comma = offsetStr.indexOf(";");
258         offsetStr = offsetStr.substring(0, comma);
259         offsetSeconds = offsetStr.toInt();
260         Serial.println("Offset seconds: ");
261         Serial.println(offsetSeconds);
262       }
263     } else {
264       Serial.println("No response from ip-api, using default IST");
265     }
266   }
267   epochTime += offsetSeconds;
268   int ss = epochTime % 60;
269   epochTime /= 60;
270   int mm = epochTime % 60;
271   epochTime /= 60;
272   int hh = epochTime % 24;
273   epochTime /= 24;
274   int days = epochTime;
275   int year = 1970;
276   int month = 1;
277   int day = 1;
278   int daysInMonth[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
279   while (days > 365) {
280     int daysThisYear = 365;
281     if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
282       daysThisYear = 366;
283     }
284     if (days >= daysThisYear) {
285       days -= daysThisYear;
286       year++;
287     } else {
288       break;
289     }
290   }
291   if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
292     daysInMonth[1] = 29;
293   }
294   for (int m = 0; m < 12; m++) {
295     if (days >= daysInMonth[m]) {
296       days -= daysInMonth[m];
297       month++;
298     } else {
299       day = days + 1;
300       break;

```

```

307 }
308 }
309 lcd.clear();
310 lcd.print("Date:");
311 lcd.setCursor(0, 1);
312 char dateStr[11];
313 sprintf(dateStr, "%04d-%02d-%02d", year, month, day);
314 lcd.print(dateStr);
315 delay(3000);
316 lcd.clear();
317 lcd.print("Time ");
318 lcd.print(":");
319 lcd.setCursor(0, 1);
320 char timeStr[9];
321 sprintf(timeStr, "%02d:%02d:%02d", hh, mm, ss);
322 lcd.print(timeStr);
323 delay(3000);
324 Serial.print("Final Datetime: ");
325 Serial.print(dateStr);
326 Serial.print(" ");
327 Serial.println(timeStr);
328 Serial.println("=== Time Display Complete ===\n");
329 }
330 bool fetchLocation(float lat, float lon) {
331   lcd.clear();
332   lcd.print("Fetching");
333   lcd.setCursor(0, 1);
334   lcd.print("Location...");
335   delay(1500);
336   client.stop();
337   delay(100);
338   Serial.println("Connecting to ipinfo.io...");
339   if (client.connect("ipinfo.io", 443)) {
340     Serial.println("Connected");
341     client.println("GET /json HTTP/1.1");
342     client.println("Host: ipinfo.io");
343     client.println("User-Agent: Arduino/1.0");
344     client.println("Connection: close");
345     client.println();
346     unsigned long timeout = millis();
347     while (!client.available()) {
348       if (millis() - timeout > 10000) {
349         Serial.println("Timeout");
350         client.stop();
351         lcd.clear();
352         lcd.print("Location timeout");
353         delay(2000);
354         return false;
355       }
356       delay(10);
357     }
358     String response = "";
359     bool jsonStarted = false;
360     while (client.available()) {
361       String line = client.readStringUntil('\n');
362       if (line.startsWith("{") {
363         jsonStarted = true;
364       }
365       if (jsonStarted) {
366         response += line;
367       }
368     }
369     client.stop();
370     Serial.println("Response received");
371     int locIndex = response.indexOf("\"loc\":");
372     if (locIndex != -1) {
373       int startQuote = response.indexOf("\"", locIndex + 6);
374       int endQuote = response.indexOf("\"", startQuote + 1);
375       String loc = response.substring(startQuote + 1, endQuote);
376       int commaPos = loc.indexOf(",");
377       if (commaPos != -1) {
378         String lats = loc.substring(0, commaPos);
379         String lons = loc.substring(commaPos + 1);
380         lat = lats.toFloat();
381         lon = lons.toFloat();
382         lcd.clear();
383         lcd.print("Latitude:");
384         lcd.setCursor(0, 1);
385         lcd.print(lat);
386         delay(3000);
387         lcd.clear();
388         lcd.print("Longitude:");
389         lcd.setCursor(0, 1);
390         lcd.print(lon);
391         delay(3000);
392         Serial.println("Location success!");
393         return true;
394       } else {
395         lcd.clear();
396         lcd.print("Parse error");
397         delay(2000);
398       }
399     } else {
400       lcd.clear();
401       lcd.print("Location failed");
402       delay(2000);
403     }
404   } else {
405     Serial.println("Connection failed!");
406     lcd.clear();
407     lcd.print("Cannot connect");
408     delay(2000);
409   }
410 }

```

```

409   return false;
410 }
411 }
412 bool enterPassword() {
413   String input = "";
414   lcd.clear();
415   lcd.print("Enter password:");
416   lcd.setCursor(0, 1);
417   while (input.length() < 4) {
418     char key = getKeyPressed_manual();
419     if (key && key >= '0' && key <= '9') {
420       input += key;
421       lcd.setCursor(input.length() - 1, 1);
422       lcd.print("*");
423       delay(300);
424     }
425   }
426   delay(800);
427   if (input == correctPassword) {
428     lcd.clear();
429     lcd.print("Password");
430     lcd.setCursor(0, 1);
431     lcd.print("verified");
432     delay(2500);
433     lcd.clear();
434     lcd.print("Access granted");
435     delay(2500);
436     return true;
437   } else {
438     lcd.clear();
439     lcd.print("Wrong password");
440     delay(2500);
441     lcd.clear();
442     lcd.print("Access denied");
443     delay(2500);
444     return false;
445   }
446 }
447 void processAlcoholLevel() {
448   int ppm = lockedAlcoholValue;
449   lcd.clear();
450   lcd.print("Analyzing...");
451   lcd.setCursor(0, 1);
452   lcd.print("Level: ");
453   lcd.print(ppm);
454   lcd.print(" ppm");
455   delay(2500);
456   float lat = 0;
457   float lon = 0;
458   if (ppm == 0) {
459     lcd.clear();
460     lcd.print("No flow detected");
461     delay(1500);
462     endTask = true;
463   }
464   else if (ppm >= 1 && ppm < SAFE_THRESHOLD) {
465     lcd.clear();
466     lcd.print("SAFE: ");
467     lcd.print(ppm);
468     lcd.print(" ppm");
469     delay(2500);
470     lcd.clear();
471     lcd.print("No alcohol");
472     lcd.setCursor(0, 1);
473     lcd.print("detected");
474     delay(2500);
475     lcd.clear();
476     lcd.print("Engine unlocked");
477     lcd.setCursor(0, 1);
478     lcd.print("twice safe");
479     logDataToSD(String(ppm) + " ppm, SAFE");
480     digitalWrite(RELAY_PIN, LOW);
481     engineUnlocked = true;
482     delay(2500);
483   }
484   else if (ppm >= SAFE_THRESHOLD && ppm <= LOW_THRESHOLD) {
485     lcd.clear();
486     lcd.print("LOW: ");
487     lcd.print(ppm);
488     lcd.print(" ppm");
489     delay(2500);
490     lcd.clear();
491     lcd.print("Range: 100-299");
492     lcd.setCursor(0, 1);
493     lcd.print("ppm");
494     delay(2500);
495     lcd.clear();
496     lcd.print("Engine unlocked");
497     lcd.setCursor(0, 1);
498     lcd.print("twice safe");
499     logDataToSD(String(ppm) + " ppm, LOW");
500     digitalWrite(RELAY_PIN, LOW);
501     engineUnlocked = true;
502     delay(2500);
503   }
504   else if (ppm > LOW_THRESHOLD && ppm <= MED_THRESHOLD) {
505     lcd.clear();
506     lcd.print("MEDIUM: ");
507     lcd.print(ppm);
508     lcd.print(" ppm");
509     delay(2500);
510     lcd.clear();
511   }

```

```

510   lcd.clear();
511   lcd.print("Range: 300-499");
512   lcd.setCursor(0, 1);
513   lcd.print("ppm");
514   delay(2500);
515   lcd.clear();
516   unsigned long startTime = millis();
517   unsigned long lastTimeUpdate = 0;
518   unsigned long lastScrollUpdate = 0;
519   int lastReading = -1;
520   const int TIMER_INTERVAL = 1000;
521   const int SCROLL_SPEED = 350;
522   const int LCD_WIDTH = 16;
523   const char* message = "Press 1 to bypass... ";
524   int messageLength = strlen(message);
525   int scrollPosition = 0;
526   bool bypassRequested = false;
527   while (millis() - startTime < 30000) {
528     unsigned long currentTime = millis();
529     char key = getKeyPressed_manual();
530     if (key == '1') {
531       bypassRequested = true;
532       break;
533     }
534     if (currentTime - lastTimeUpdate >= TIMER_INTERVAL) {
535       lastTimeUpdate = currentTime;
536       int remaining = 30 - ((currentTime - startTime) / 1000);
537       lcd.setCursor(0, 1);
538       lcd.print("Time: ");
539       lcd.print(remaining);
540       lcd.print("s ");
541     }
542     if (currentTime - lastScrollUpdate >= SCROLL_SPEED) {
543       lastScrollUpdate = currentTime;
544       lcd.setCursor(0, 0);
545       for (int i = 0; i < LCD_WIDTH; i++) {
546         int charIndex = (scrollPosition + i) % messageLength;
547         lcd.print(message[charIndex]);
548       }
549       scrollPosition++;
550       if (scrollPosition >= messageLength) {
551         scrollPosition = 0;
552       }
553     }
554   }
555   if (bypassRequested) {
556     lcd.clear();
557     lcd.print("Bypass required");
558     delay(2500);
559     lcd.clear();
560     lcd.print("Press 1 for");
561     lcd.setCursor(0, 1);
562     lcd.print("password");
563     unsigned long pStartTime = millis();
564     bool starPressed = false;
565     while (millis() - pStartTime < 15000) {
566       char key = getKeyPressed_manual();
567       if (key == "*") {
568         starPressed = true;
569         break;
570       }
571     }
572     if (starPressed && enterPassword()) {
573       lcd.clear();
574       lcd.print("Engine unlocked");
575       delay(2500);
576       lcd.clear();
577       lcd.print("Drive carefully");
578       logDataToSD(String(ppm) + " ppm, MEDIUM (Bypassed)");
579       digitalWrite(RELAY_PIN, LOW);
580       tone(BUZZER_PIN, 2000);
581       engineUnlocked = true;
582       delay(2500);
583       displayDataTime();
584       if (fetchLocation(lat, lon) {
585         Serial.println("Location fetched. Saving and uploading.");
586         sendThingSpeak(lat, lon, ppm);
587       } else {
588         Serial.println("Failed to thing...");
589       }
590     } else {
591       lcd.clear();
592       lcd.print("Bypass failed");
593       delay(2500);
594       lcd.clear();
595       lcd.print("Engine locked");
596       logDataToSD(String(ppm) + " ppm, MEDIUM (Bypass FAILED)");
597       digitalWrite(RELAY_PIN, HIGH);
598       tone(BUZZER_PIN, 3000, 30000);
599       delay(2500);
600       displayDataTime();
601       if (fetchLocation(lat, lon) {
602         Serial.println("Location fetched. Saving and uploading.");
603         sendThingSpeak(lat, lon, ppm);
604       } else {
605         Serial.println("Failed to thing...");
606       }
607     }
608     delay(2000);
609     lcd.clear();
610     lcd.print(" --End-- ");
611     delay(2000);
612     endTask = true;
613   }
614   else if (ppm >= MED_THRESHOLD) {
615     lcd.clear();
616     lcd.print("No response");
617     delay(2500);
618     lcd.clear();
619     lcd.print("Engine locked");
620     logDataToSD(String(ppm) + " ppm, MEDIUM (No Bypass)");
621     digitalWrite(RELAY_PIN, HIGH);
622     displayDataTime();
623     if (fetchLocation(lat, lon) {
624       Serial.println("Location fetched. Saving and uploading.");
625       sendThingSpeak(lat, lon, ppm);
626     } else {
627       Serial.println("Failed to thing...");
628     }
629     delay(2000);
630     endTask = true;
631   }
632   else if (ppm >= MED_THRESHOLD) {

```



```

613 } else {
614   lcd.clear();
615   lcd.print("No response");
616   delay(2500);
617   lcd.clear();
618   lcd.print("Engine locked");
619   digitalWrite(RELAY_PIN, HIGH);
620   digitalWrite(BUZZER_PIN, HIGH);
621   digitalWrite(LED_PIN, HIGH);
622   if (switchLocation(lat, lon)) {
623     Serial.println("Location fetched. Saving and uploading.");
624     sendToThingSpeak(lat, lon, ppm);
625   } else {
626     Serial.println("Failed to thing...");
627     delay(2000);
628     endtask = true;
629   }
630 }
631
632 else if (ppm > MID_THRESHOLD) {
633   lcd.clear();
634   lcd.print("MID: ");
635   lcd.print(ppm);
636   lcd.print(" ppm");
637   delay(2500);
638   lcd.clear();
639   lcd.print("Bypass above");
640   lcd.setCursor(0, 1);
641   lcd.print("500 ppm");
642   delay(2500);
643   lcd.clear();
644   lcd.print("Bypass denied");
645   delay(2500);
646   lcd.clear();
647   lcd.print("Engine locked");
648   digitalWrite(RELAY_PIN, HIGH);
649   digitalWrite(BUZZER_PIN, HIGH);
650   digitalWrite(LED_PIN, HIGH);
651   delay(2500);
652   digitalWrite(LED_PIN, LOW);
653   if (switchLocation(lat, lon)) {
654     Serial.println("Location fetched. Saving and uploading.");
655     sendToThingSpeak(lat, lon, ppm);
656   } else {
657     Serial.println("Failed to thing...");
658     delay(2000);
659     lcd.clear();
660     lcd.print(" --End--");
661   }
662 }
663
664 void logDataToSD(String data) {
665   File dataFile = SD.open("dt.txt", FILE_WRITE);
666   if (dataFile) {
667     dataFile.println(logCounter + " " + data);
668     logCounter++;
669     dataFile.close();
670   } else {
671     Serial.println("Error opening dt.txt");
672     lcd.clear();
673     lcd.print("SD Write ERROR");
674     delay(2000);
675   }
676 }
677
678 void displayingData() {
679   lcd.clear();
680   File dataFile = SD.open("dt.txt");
681   if (!dataFile) {
682     if (dataFile) {
683       dataFile.close();
684     }
685     Serial.println("Error opening dt.txt or file not found");
686     lcd.print("log is Empty");
687     delay(2500);
688     f = false;
689     return;
690   }
691   if (dataFile.size() == 0) {
692     if (dataFile) {
693       dataFile.close();
694     }
695     Serial.println("log file is empty.");
696     lcd.print("log is Empty");
697     dataFile.close();
698     delay(2500);
699     f = false;
700     return;
701   }
702   lcd.print("Past Records: ");
703   lcd.setCursor(0, 1);
704   lcd.print("A-Dest, -Exit");
705   delay(2000);
706   while (true) {
707     if (dataFile.available()) {
708       String line = dataFile.readStringUntil('\n');
709       line.trim();
710       Serial.println("Displaying: " + line);
711       lcd.clear();
712       if (line.length() < 10) {
713         lcd.print(line);
714       } else {
715         lcd.print(line.substring(0, 10));
716         lcd.setCursor(0, 1);
717         lcd.print(line.substring(10));
718       }
719       char key = 0;
720       while (key != 'A' && key != '*') {
721         key = getKeyPressed_manual();
722         delay(20);
723       }
724       if (key == '*') {
725         Serial.println("Exiting viewer.");
726         break;
727       }
728       if (key == 'A') {
729         Serial.println("Next entry.");
730         continue;
731       }
732     } else {
733       Serial.println("End of log reached.");
734       lcd.clear();
735       lcd.print("End of log");
736     }
737   }
738 }

```

```

714 } else {
715   lcd.print(line.substring(0, 10));
716   lcd.setCursor(0, 1);
717   lcd.print(line.substring(10));
718 }
719 char key = 0;
720 while (key != 'A' && key != '*') {
721   key = getKeyPressed_manual();
722   delay(20);
723 }
724 if (key == '*') {
725   Serial.println("Exiting viewer.");
726   break;
727 }
728 if (key == 'A') {
729   Serial.println("Next entry.");
730   continue;
731 }
732 } else {
733   Serial.println("End of log reached.");
734   lcd.clear();
735   lcd.print("End of log");
736   lcd.setCursor(0, 1);
737   lcd.print("Press * to exit");
738   char key = 0;
739   while (key != '*' && key != 'A') {
740     key = getKeyPressed_manual();
741     delay(20);
742   }
743   Serial.println("Exiting viewer.");
744   break;
745 }
746 }
747 dataFile.close();
748 decide();
749 }
750 void setup() {
751   pinMode(RELAY_PIN, OUTPUT);
752   pinMode(BUZZER_PIN, OUTPUT);
753   digitalWrite(RELAY_PIN, HIGH);
754   for (int i = 0; i < ROWS; i++) {
755     pinMode(rowPin[i], OUTPUT);
756     digitalWrite(rowPin[i], HIGH);
757   }
758   for (int i = 0; i < COLS; i++) {
759     pinMode(colPin[i], INPUT_PULLUP);
760   }
761   Serial.begin(9600);
762   lcd.begin(16, 2);
763   lcd.backlight();
764   Serial.println("initializing SD card...");
765   if (!SD.begin(SD_CS_PIN)) {
766     Serial.println("Card failed, or not present");
767     lcd.clear();
768     lcd.print("SD Card ERROR");
769     lcd.setCursor(0, 1);
770     lcd.print("Check wiring");
771     delay(2000);
772   } else {
773     Serial.println("SD card initialized.");
774   }
775   initializeSystem();
776   lcd.clear();
777   WiFi.begin(ssid, pass);
778   unsigned long wifiStart = millis();
779   while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < 15000) {
780     delay(500);
781     Serial.print(".");
782   }
783   if (WiFi.status() == WL_CONNECTED) {
784     Serial.println("WiFi connected");
785     Serial.print("IP: ");
786     Serial.println(WiFi.localIP());
787   } else {
788     lcd.clear();
789     lcd.print("WiFi Failed");
790     lcd.setCursor(0, 1);
791     lcd.print("Continuing...");
792     delay(2500);
793     Serial.println("WiFi failed!");
794   }
795   lcd.clear();
796   lcd.print("Follow following");
797   lcd.setCursor(0, 1);
798   lcd.print("Procedures: ");
799   delay(3000);
800   decide();
801   waitForBlow();
802   processAlcoholLevel();
803 }
804 void loop() {
805   if (engineUnlocked) {
806     lcd.clear();
807     bool vehicleRunning = false;
808     while (engineUnlocked) {
809       lcd.clear();
810       lcd.print("Vehicle started");
811       lcd.setCursor(0, 1);
812       lcd.print("Drive safely");
813       vehicleRunning = true;
814       delay(3000);
815       lcd.clear();
816       lcd.print("Press D to turn");
817       lcd.setCursor(0, 1);
818       lcd.print("off engine");
819       while (true) {
820         char key = getKeyPressed_manual();
821         if (key == 'D') {
822           digitalWrite(RELAY_PIN, HIGH);
823           noTone(BUZZER_PIN);
824           break;
825         }
826       }
827       lcd.clear();
828       lcd.print("Engine OFF");
829       delay(2500);
830       engineUnlocked = false;
831       vehicleRunning = false;
832       lockedAlcoholValue = -1;
833     }
834     lcd.clear();
835     lcd.print("Restarting the");
836     lcd.setCursor(0, 1);
837     lcd.print("system...");
838     delay(2500);
839     decide();
840     waitForBlow();
841     processAlcoholLevel();
842   } else if (endtask) {
843     lcd.clear();
844     lcd.print("Restarting the");
845     lcd.setCursor(0, 1);
846     lcd.print("system...");
847     delay(2500);
848     decide();
849     waitForBlow();
850     processAlcoholLevel();
851   }
852 }

```

```

796   lcd.print("Follow following");
797   lcd.setCursor(0, 1);
798   lcd.print("Procedures: ");
799   delay(3000);
800   decide();
801   waitForBlow();
802   processAlcoholLevel();
803 }
804 void loop() {
805   if (engineUnlocked) {
806     lcd.clear();
807     bool vehicleRunning = false;
808     while (engineUnlocked) {
809       lcd.clear();
810       lcd.print("Vehicle started");
811       lcd.setCursor(0, 1);
812       lcd.print("Drive safely");
813       vehicleRunning = true;
814       delay(3000);
815       lcd.clear();
816       lcd.print("Press D to turn");
817       lcd.setCursor(0, 1);
818       lcd.print("off engine");
819       while (true) {
820         char key = getKeyPressed_manual();
821         if (key == 'D') {
822           digitalWrite(RELAY_PIN, HIGH);
823           noTone(BUZZER_PIN);
824           break;
825         }
826       }
827       lcd.clear();
828       lcd.print("Engine OFF");
829       delay(2500);
830       engineUnlocked = false;
831       vehicleRunning = false;
832       lockedAlcoholValue = -1;
833     }
834     lcd.clear();
835     lcd.print("Restarting the");
836     lcd.setCursor(0, 1);
837     lcd.print("system...");
838     delay(2500);
839     decide();
840     waitForBlow();
841     processAlcoholLevel();
842   } else if (endtask) {
843     lcd.clear();
844     lcd.print("Restarting the");
845     lcd.setCursor(0, 1);
846     lcd.print("system...");
847     delay(2500);
848     decide();
849     waitForBlow();
850     processAlcoholLevel();
851   }
852 }

```